INF384 · TEMAS AVANZADOS EN INGENIERÍA DE SOFTWARE 1 · 2026-2

Sesión 06

Despliegue continuo: estrategias, aprobaciones y validación

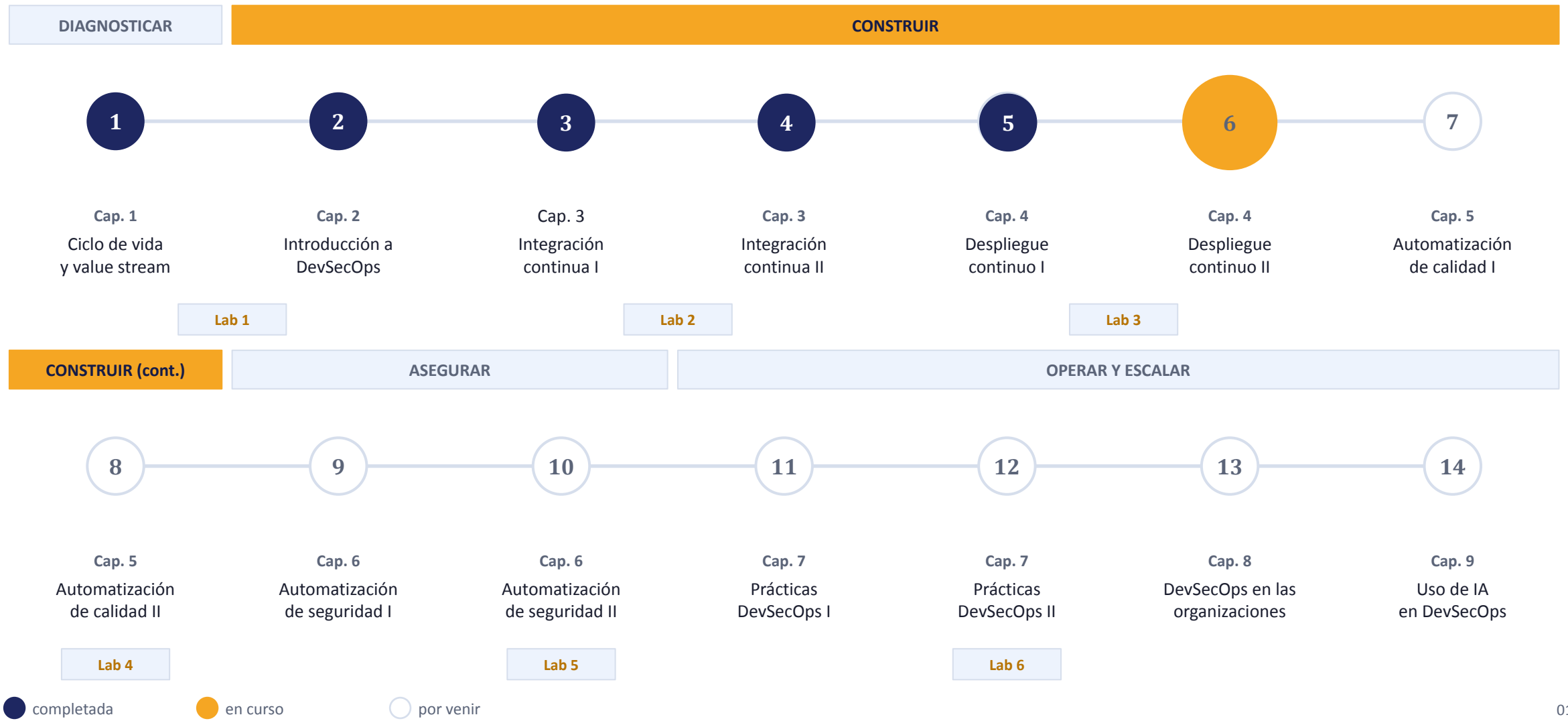
Estrategias de despliegue, compuertas de aprobación y la validación que confirma o revierte

---

Capítulo 4 (2 de 2) · 3 horas | RA2 · RA4 | Prof. José Felipe del Castillo Salazar

INF384 · 9 CAPÍTULOS · 14 SESIONES · 42 HORAS

La ruta del curso



SESIÓN 6

Objetivo de la sesión y ruta

0 **Laboratorio 3**

1 **Estrategias de despliegue**

2 **Flujo de aprobaciones**

3 **Observabilidad**

Resultados de aprendizaje vinculados: RA2 y RA4 — observabilidad, mejora continua y gestión de incidentes.

LABORATORIO 3 · RESOLUCIÓN

Lab 3 · Pipeline de aplicación (app.yml)



DOCKERFILE · 5 DEFECTOS MARCADOS + 1 DEDUCIDO

Defecto	Antes	Después
1 Versión flotante	`FROM lambda/nodejs:latest`	`public.ecr.aws/lambda/nodejs:20`
2 Copia todo	`COPY . .`	lock file, luego `src` (caché)
3 No reproducible	`npm install`	`npm ci` desde el lock file
4 Credencial en la imagen	`ENV DB_PASSWORD=...`	eliminada (visible en `docker history`)
5 Tools en runtime	`dnf install procps-ng vim`	eliminado
+ Una sola etapa	—	`build` (esbuild) → `runtime` solo con `dist/handler.js`

+ `.dockerignore` (`.git`, `node_modules`, `infra`, `test`, `coverage`…). **Aceptación:** `docker history` sin credenciales; etapa final sin `node_modules` ni `dnf install` (el binario viene en la base: se evalúa que no se invoque).

LAS CUATRO COMPUERTAS

Job	Herramienta	¿Detiene?	Detalle clave
`calidad`	SonarCloud `sonarqube-scan-action@v8`	No	cobertura del job `pruebas` (artifact); `fetch-depth: 0` para distinguir el código nuevo
`secretos`	`gitleaks-action@v3`	**Sí**	`fetch-depth: 0` la credencial sigue viva en el historial aunque se borre del último commit
`dependencias`	`trivy-action@v0.36.0` (fs, `exit-code: 0`)	No	`TRIVY_INCLUDE_DEV_DEPS=true` árbol completo del lock file
`estatico`	`semgrep/semgrep` con `docker run`	No	`p/default` + `p/javascript` sobre `src/`

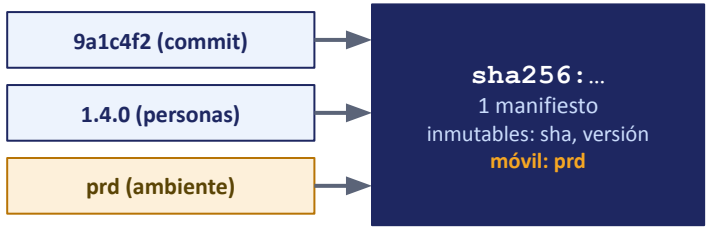
Las tres informativas dejan rastro visible: un `::warning::` por hallazgo y una tabla en `GITHUB_STEP_SUMMARY`. El hallazgo se ve sin abrir el log.

EXCEPCIÓN = 4 DATOS, SIEMPRE

identificador (el de LA herramienta: `CVE-2023-26136` en Trivy, `GHSA-72xf-g2v4-qvf3` en otra) · **razón** · **responsable** · **vencimiento**. Junto al código o en el paso que lo detecta. Una excepción sin vencimiento silencia el hallazgo para siempre. **Una credencial nunca se exceptúa:** se retira, se limpia del historial y se rota.

PUBLICAR · NEEDS: 4 COMPUERTAS · SOLO MAIN

`docker build --platform linux/amd64` + label OCI `image.title`
Push **solo** con el tag `sha` (7 caracteres del commit)
`version` (`git describe`, nunca a mano) y `prd` → mismo manifiesto vía `batch-get-image` + `put-image`
Output `digest`: `desplegar` actualiza la Lambda **por digest**



Lab 3 · Pipeline de infraestructura (infra.yml)

validar

DETIENE

```
terraform fmt -check -recursive
terraform init -backend-config=backend.hcl
terraform validate
```

Necesita credenciales AWS: `init` habla con el backend en S3. `backend.hcl` se versiona: el nombre del bucket no es secreto.

politicas

DETIENE

`trivy-action@v0.36.0` · `scan-type: config` · `scan-ref: infra/`

`severity: HIGH,CRITICAL` + `exit-code: 1`: umbral declarado en el workflow

Sin credenciales: analiza el código declarado

EXCEPCIÓN PEGADA AL RECURSO

```
# razón · responsable
#trivy:ignore:<ID>:exp:2026-10-31
```

Pasada la fecha, el hallazgo vuelve a contar y el pipeline se detiene solo.

plan + compuerta

```
terraform plan -out=tfplan
terraform show -json tfplan > tfplan.json
```

COMPUERTA-PLAN.JS · NÚCLEO

```
const plan = JSON.parse(fs.readFileSync(process.env.RUTA_PLAN));
```

La compuerta evalúa el efecto calculado en `tfplan.json`: el mismo `.tf`

aplicar

Solo en `main`, `needs: plan`

Descarga el artifact `tfplan` y ejecuta `terraform apply tfplan` sin recalcular

Cambio de la sesión: logs 1 → 7 días, memoria 512 → 1024 MB. Update in-place: pasa la compuerta sin autorización

LA IMAGEN ES DE LA APP

```
lifecycle {
  ignore_changes = [image_uri]
}
```

Sin esta línea, cada `apply` de infra revierte la Lambda a la imagen `bootstrap`.

LABORATORIO 3 · RESOLUCIÓN

Lab 3 · Inyecciones de falla y evidencia

Inyección 1 · credencial en el código

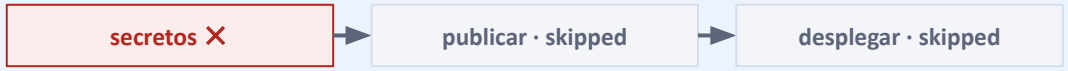
→ secretos

Qué se hace — Escribir en `src/` una AWS key con forma realista: `AKIA` + 16 caracteres de `[A-Z2-7]`.

Trampa — `AKIAIOSFODNN7EXAMPLE` está en la allowlist de gitleaks: no produce hallazgo y la compuerta parece rota.

Esperado — `secretos` en rojo, `publicar` y `desplegar` en skipped, ninguna imagen nueva en ECR.

Cierre — Revertir, limpiar el historial y rotar. Inyectar una dependencia vulnerable no prueba nada: esa compuerta solo informa.



Inyección 2 · destrucción en el plan

→ compuerta del plan

Qué se hace — Renombrar el log group (`name = "/aws/lambda/..."`). Terraform lo resuelve como replace = delete + create.

Esperado — Plan publicado en el PR; `plan` falla con «El plan destruye 1 recurso(s) y nadie lo autorizó»; `aplicar` no se ejecuta.

SALIDA DEL PLAN

```
# aws_cloudwatch_log_group.funcion must be replaced
-/+ resource "aws_cloudwatch_log_group" "funcion" {
      ~ name = "/aws/lambda/…" -> "/aws/lambda/…" # forces replacement
```

Un pipeline que solo se ha visto pasar no está probado.

QUÉ DEBE MOSTRAR LA EVIDENCIA

- ✓ Los dos pipelines en verde (tras revertir las inyecciones)
- ✓ 3 tags → 1 mismo digest en ECR
- ✓ Tamaño etapa build vs runtime
- ✓ Salida de la Lambda con la versión
- ✓ Excepciones anotadas (4 datos) y dónde están
- ✓ Plan publicado en el PR del cambio 7 días / 1024 MB
- ✓ Evidencia de las 2 inyecciones

CONTINUIDAD CON LA SESIÓN 5

Dónde estamos en el pipeline

Commit	Build	Calidad	Seguridad	Publicar	Aprobar	Desplegar	Observar

Lo que la sesión 5 dejó resuelto

- Existe un artefacto único, versionado e identificable por su huella.
- La configuración vive fuera del artefacto y se inyecta en el despliegue.
- El ambiente está declarado como código y se puede reconstruir.
- El plan de IaC muestra la diferencia antes de aplicarla.

Lo que todavía no está decidido

- ¿El servicio se detiene mientras desplegamos la versión nueva?
- ¿Cuántos usuarios ven la versión nueva antes de confirmar que funciona?
- ¿Quién autoriza el paso a producción y con qué información decide?
- ¿Cómo se sabe que salió bien, y en cuánto tiempo se sabe?

Un despliegue automatizado sin validación que leer, automatiza también el error. Entregamos más rápido un error y a más usuarios

A

Estrategias de despliegue

Cómo entra la versión nueva y cuánta exposición tiene el usuario durante la transición

Desplegar y publicar son dos eventos separables

Desplegar

Llevar el artefacto al ambiente y ponerlo a ejecutar. Es una operación técnica, la ejecuta el pipeline y su riesgo es de disponibilidad.

Publicar

Poner la funcionalidad a disposición del usuario. Es una decisión de producto, la toma alguien del negocio y su riesgo es de experiencia.

1

Feature Flag separa los dos eventos

Ya la vieron en la sesión 3 como habilitador de ramas cortas. Acá cumple la segunda función: desacoplar el despliegue de la exposición.

2

Las estrategias de despliegue resuelven el riesgo técnico

Controlan cuántas instancias y cuántos usuarios reciben la versión nueva mientras se verifica que funciona.

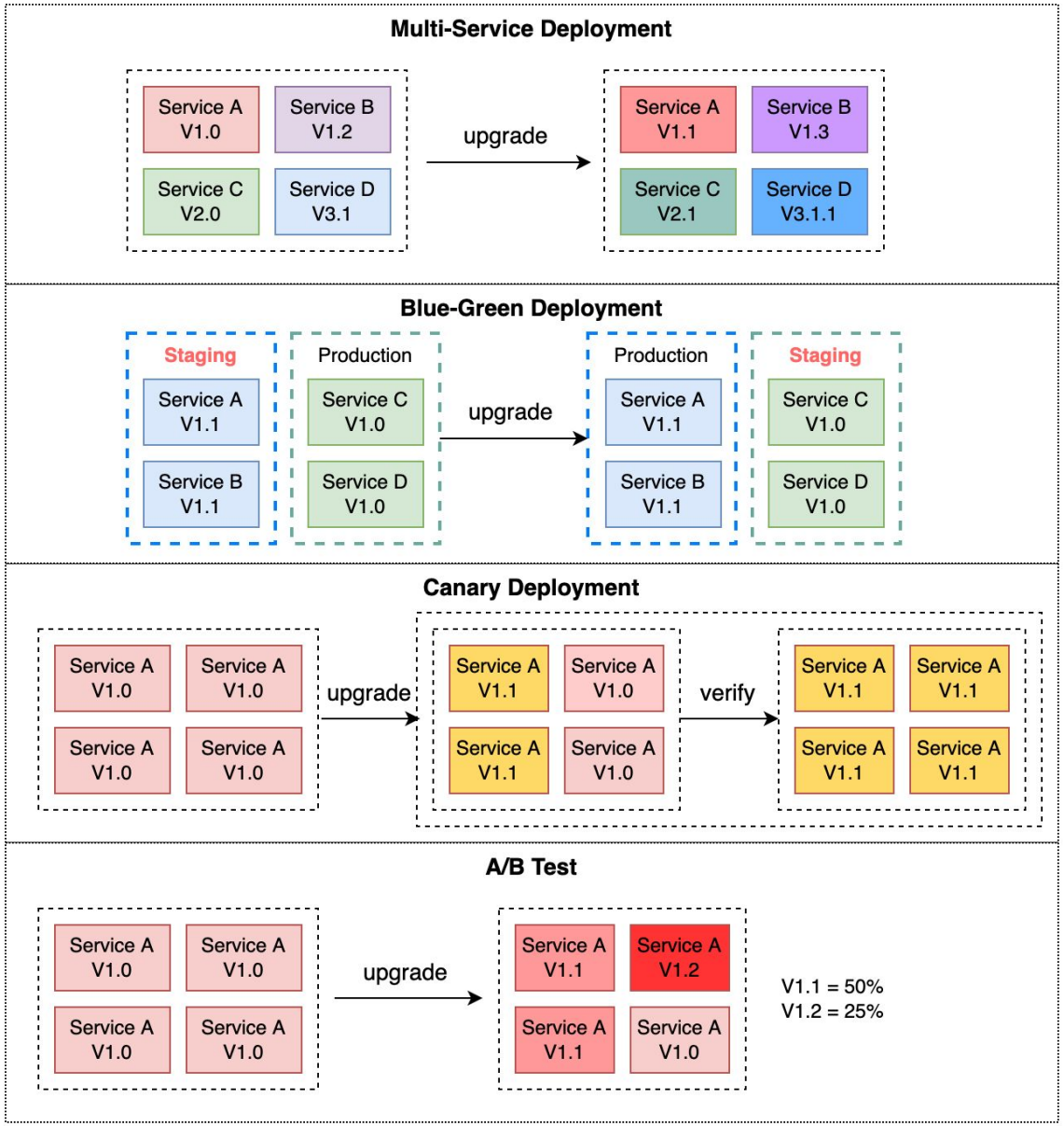
3

Las dos capas se combinan

Un despliegue gradual de código que además llega con la funcionalidad apagada da dos mecanismos de reversión independientes.

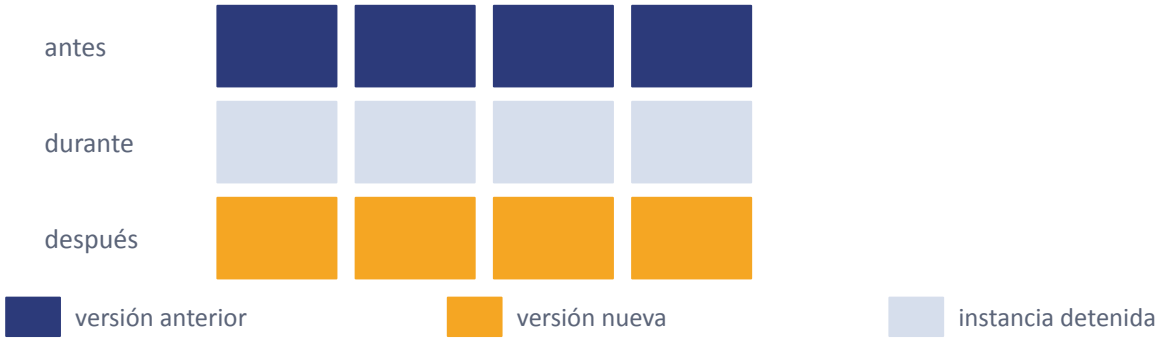
Apagar una bandera toma segundos y no requiere pipeline. Revertir un despliegue toma minutos y sí lo requiere.

Estrategias de Despliegue



Reemplazo total y reemplazo progresivo

Reemplazo total



Qué implica

Hay una ventana sin servicio, del tamaño del tiempo de arranque.
No conviven dos versiones: no hay restricción sobre el esquema de datos ni sobre la compatibilidad entre versiones.
Revertir exige repetir la operación completa, con una segunda ventana de indisponibilidad.

Reemplazo progresivo



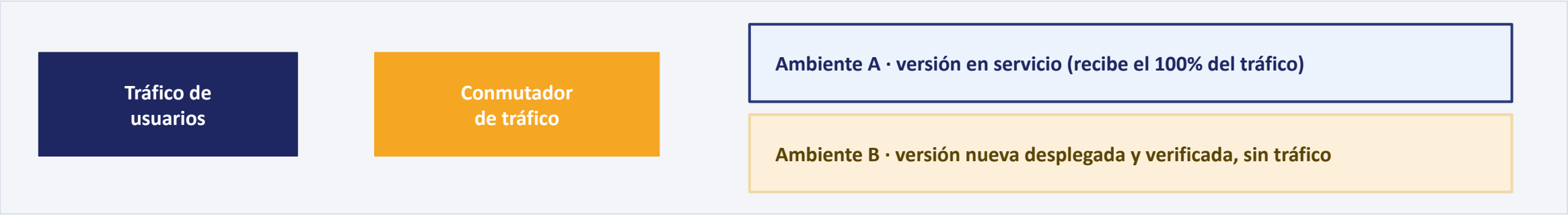
Qué implica

No hay ventana sin servicio: las instancias se reemplazan por tandas.
Las dos versiones atienden usuarios al mismo tiempo, y tienen que ser compatibles entre sí y con los datos.
Revertir es otra operación progresiva, y mientras ocurre el sistema sigue mezclando versiones.

El reemplazo progresivo es el comportamiento por defecto de la mayoría de orquestadores

CAPÍTULO 4.3 · ESTRATEGIA 3

Dos ambientes completos y un conmutador



1. **La versión nueva se verifica con tráfico real cero**
2. **La transición es instantánea y la reversión también**
3. **Cuesta el doble de infraestructura durante la transición**
4. **Las sesiones activas y los datos no se conmutan**

Es la estrategia con la reversión más rápida y el costo de infraestructura más alto. La decisión se toma comparando ese costo contra el costo de un minuto caído.

Exposición gradual con decisión basada en señal

5% del tráfico	25% del tráfico	50% del tráfico	100% del tráfico
observación 10 min	observación 20 min	observación 30 min	señal estable en cada paso

1

La condición de avance se escribe antes de empezar

Tasa de error, latencia y saturación con umbral y ventana declarados. Es el mismo diseño del quality gate del Capítulo 3, sobre datos de producción.

2

Sin observabilidad la estrategia no existe

Exponer al 5% y mirar un tablero durante diez minutos sin umbral definido es desplegar igual que siempre, con más pasos.

3

El volumen tiene que alcanzar para decidir

Con poco tráfico, el 5% no produce suficientes muestras para distinguir una falla de la variación normal. La estrategia deja de aportar información.

4

La reversión automática es la contraparte obligatoria

Si el umbral se rompe, el tráfico vuelve a la versión anterior sin esperar a que alguien lo apruebe. Es lo que baja el tiempo de restauración.

El despliegue canario limita el daño al porcentaje expuesto y acorta el tiempo hasta detectarlo. Las dos cosas dependen de la señal, no de la herramienta.

Criterios para elegir la estrategia

Criterio	Reemplazo total	Progresivo	Dos ambientes	Canario
Indisponibilidad	Sí	No	No	No
Convivencia de versiones	No	Sí	Durante la transición	Sí, prolongada
Costo de infraestructura	Bajo	Bajo	Alto temporal	Medio
Velocidad de reversión	Lenta	Media	Inmediata	Inmediata
Exige observabilidad	Deseable	Deseable	Necesaria	Indispensable
Usuarios expuestos al fallo	Todos	Parte creciente	Todos tras conmutar	Porcentaje acotado

La estrategia se elige por el costo de un fallo en ese servicio y por la capacidad real de detectarlo

El esquema de datos es el límite real de todas las estrategias

Una migración que elimina o renombra una columna rompe la versión anterior, y con ella cualquier posibilidad de revertir.

El código se revierte en minutos; el esquema de datos, no.

1

Expandir

Se agrega la estructura nueva sin tocar la existente. Las dos versiones del código funcionan contra el mismo esquema.

2

Migrar y escribir en ambas

La versión nueva escribe en las dos estructuras mientras se copian los datos históricos. La anterior sigue operando sin enterarse.

3

Trasladar la lectura

Cuando los datos están completos, la versión nueva lee de la estructura nueva. Sigue siendo posible volver atrás.

4

Eliminar versión anterior

Solo cuando ninguna versión desplegada usa la estructura antigua, se elimina. Es un cambio propio, con su propio despliegue.

Toda migración se diseña compatible hacia atrás con la versión anterior en ejecución. Es la condición que hace posible desplegar sin detener el servicio.

¿Reversar o Corregir?

Volver a la versión anterior

Se despliega el artefacto anterior, que ya existe en el registro y ya fue validado. El tiempo es el de un despliegue.

Requisito: el artefacto anterior sigue disponible y el esquema de datos lo admite.

Es la opción por defecto durante un incidente.

Avanzar con una corrección

Se escribe la corrección y se despliega una versión nueva. El tiempo incluye diagnosticar, corregir y recorrer el pipeline completo.

Requisito: un pipeline rápido y confiable, porque corre bajo presión.

Es la única opción cuando el esquema de datos ya cambió.

1

La decisión se toma antes del incidente, no durante

Qué condición dispara la reversión y quién puede ejecutarla se define en frío. A las dos de la madrugada no se diseña política.

2

La reversión se ensaya

Un mecanismo de reversión que nunca se ejecutó es una suposición.

El tiempo de restauración del servicio se define por la velocidad de la reversión, no por la velocidad del diagnóstico.

B

Flujo de aprobaciones

Quién autoriza el paso a producción, con qué información decide y con qué credenciales opera el pipeline

Qué aporta una aprobación y qué no

Lo que una aprobación sí resuelve

Segregación de funciones: quien escribe el cambio no es quien lo autoriza.

Registro auditable de quién decidió, cuándo y sobre qué versión exacta.

Coordinación con el negocio cuando el cambio tiene fecha o ventana acordada.

Lo que una aprobación no resuelve

No detecta defectos: quien aprueba no ejecuta el código ni revisa las pruebas.

No reduce el riesgo del cambio (por sí solo); lo traslada de fecha y lo acumula con otros.

No sustituye una validación automática ausente, aunque se sienta así.

1

La evidencia del sector va en contra de la intuición

"Accelerate" reporta que la aprobación de un comité externo al equipo no mejora la estabilidad y sí alarga el tiempo de entrega. Es bibliografía del curso.

2

La alternativa no es eliminar el control, es cambiar su forma

Revisión por pares dentro del equipo, validación automática y despliegue reversible producen la misma evidencia de control sin la espera.

3

En contextos regulados la aprobación se conserva y se automatiza la evidencia

El pipeline produce el registro de qué versión se aprobó, quién la aprobó y qué validaciones pasó, sin trabajo manual adicional.

Una aprobación que nadie puede rechazar en la práctica es un paso de espera con apariencia de control.

El ambiente protegido como compuerta

Revisores requeridos

El trabajo de despliegue se detiene hasta que una persona o equipo designado lo autoriza. Queda registrado quién fue.

Espera obligatoria

Un tiempo mínimo entre la aprobación y la ejecución, para que una aprobación apresurada se pueda cancelar.

Restricción de rama

Solo la rama principal puede desplegar a producción. Cierra el camino de desplegar desde una rama sin validar.

Secretos por ambiente

Las credenciales de producción solo existen dentro de ese ambiente. Un trabajo que apunta a certificación no puede alcanzarlas.

1

La protección vive en la plataforma, no en el archivo del pipeline

Quien edita el archivo del pipeline no puede levantarse el GATE a sí mismo. Ahí está la segregación real.

2

Quien aprueba necesita ver qué está aprobando

Versión, diferencia respecto de lo desplegado y resultado de las validaciones. Una aprobación sin esa información es una firma.

Con qué credenciales despliega el pipeline

Llave estática guardada como secreto

Vive hasta que alguien la rota, y casi nadie la rota.

Quien la extrae una vez la puede usar desde cualquier lugar y en cualquier momento.

Se copia a ambientes locales para pruebas y deja de estar en un solo sitio.

Su alcance suele ser mucho mayor que lo que el pipeline necesita.

Identidad federada de corta duración

El pipeline presenta un token firmado que declara repositorio, rama y ejecución.

La nube lo verifica y entrega credenciales que expiran en minutos.

No hay secreto permanente que robar, ni que rotar.

La confianza se declara para un repositorio y una rama específicos.

1

El permiso se acota a lo que ese trabajo necesita

Publicar una imagen y actualizar un servicio. No administrar la cuenta. El pipeline es superficie de ataque, como ya se dijo en la sesión 4.

2

El entorno académico del curso no permite esta configuración

Sus credenciales son temporales, se renuevan por sesión y hay que volver a cargarlas. La limitación es parte del aprendizaje del Laboratorio 3.

Una llave de despliegue con permisos amplios y sin vencimiento es el hallazgo de seguridad más caro que puede tener un repositorio.

Anatomía del trabajo de despliegue

GITHUB ACTIONS — DESPLIEGUE CON VERIFICACIÓN

```
jobs:
  desplegar:
    needs: publicar
    if: github.ref == 'refs/heads/main'
    environment: produccion            # 1
    steps:
      - name: Obtener credenciales
        uses: <accion-de-credenciales>  # 2
      - name: Registrar la version nueva
        run: <registrar-definicion>
             --imagen $REG/app@$DIGEST  # 3
      - name: Actualizar el servicio
        run: <actualizar-servicio>
      - name: Esperar estabilidad
        run: <esperar-servicio-estable> # 4
      - name: Prueba de humo
        run: curl -fsS $URL/salud
      - name: Revertir si fallo
        if: failure()                  # 5
        run: <volver-a-la-revision-previa>
```

1. **La compuerta de aprobación**
   Declarada en la plataforma, no en el archivo del pipeline.
2. **Credenciales de corta duración**
   Se obtienen al inicio del trabajo y expiran con él.
3. **Se despliega por huella, no por etiqueta**
   Identifica el artefacto exacto que pasó las validaciones.
4. **El pipeline espera el resultado real**
   Sin esta espera termina antes de saber si el servicio arrancó.
5. **La reversión es parte del mismo trabajo**
   Si la verificación falla, la versión anterior vuelve sola.

C

Observabilidad

Las señales que permiten afirmar que el despliegue salió bien, y las que disparan la reversión

OBSERVABILIDAD

Métricas

¿Hay un problema?

Series numéricas agregadas en el tiempo: tasa de error, latencia, saturación de recursos. Baratas de guardar, responden rápido y no dicen por qué.

Trazas

¿Dónde está el problema?

El recorrido de una petición por todos los servicios que la atendieron, con el tiempo que pasó en cada uno. Localizan el componente responsable.

Registros / Logs

¿Qué exactamente ocurrió?

El detalle de un evento puntual. Caros en volumen y de poco uso si no se pueden correlacionar con la petición que los originó.

1

El identificador de correlación es lo que vuelve útil al conjunto

Sin un identificador común que atraviese los servicios, las tres señales son tres archivos que nadie puede cruzar durante un incidente.

2

La versión desplegada se emite como atributo de cada señal

Es lo que permite afirmar que el error aparece en la versión nueva y no en la anterior. Sin eso, la comparación es una impresión.

El punto 2 es lo que conecta este bloque con el Capítulo 4 completo: la trazabilidad del artefacto termina siendo un atributo de la señal en producción.

El despliegue se verifica con una condición, igual que el build



1

La verificación técnica no es la de negocio

Que el proceso arranque y responda no prueba que la funcionalidad haga lo correcto. Son dos comprobaciones distintas con dos umbrales distintos.

2

La ventana de observación se declara antes

Duración, métrica y umbral escritos en el repositorio. La misma disciplina del quality gate, aplicada al comportamiento en producción.

3

La reversión automática se ejecuta sin consultar

Si esperar una aprobación es parte del camino de reversión, el tiempo de restauración incluye el tiempo de encontrar a esa persona.

4

El despliegue fallido se cuenta

La proporción de despliegues que requieren reversión o corrección urgente es la tasa de fallo de cambios de la sesión 1.

Objetivo de nivel de servicio (SLA, SLO)

1

Indicador

Una medición concreta de la experiencia del usuario: proporción de peticiones atendidas por debajo de un tiempo dado.

2

Objetivo

El valor acordado para ese indicador en una ventana de tiempo. Un objetivo del 100% es una declaración de intenciones sin costo asociado.

3

Presupuesto de error

La diferencia entre el objetivo y la perfección. Es la cantidad de fallo que la organización aceptó gastar, y se gasta desplegando.

4

La regla de consumo es la que da valor al acuerdo

Mientras queda presupuesto, el equipo despliega a su ritmo. Cuando se agota, la prioridad pasa a estabilidad hasta recuperarlo.

El presupuesto de error convierte la discusión entre velocidad y estabilidad en un número acordado por adelantado, en lugar de una negociación por incidente.

SLA

Customer Promise

Promise
99.5% uptime

Written by
The Legal Department and Product Managers

Written for
The customer and setting SLOs

SLO

Internal Goal

Goal
99.9% uptime

Written by
Product Managers to meet SLAs

Written for
Internal teams to meet SLA requirements

SLI

Actual Performance

Performance
99.7% uptime

Monitored by
Tools like Prometheus and SolarWinds

Used by
Customers and teams to determine if SLAs and SLOs were met

Puntos Clave

1. **Desplegar y publicar se pueden separar**
   La bandera de funcionalidad y la estrategia de despliegue son dos mecanismos de reversión independientes.

2. **La estrategia se elige por el costo del fallo y la capacidad de detectarlo**
   El nombre de la estrategia no indica madurez; la señal disponible sí indica viabilidad.

3. **El esquema de datos es el límite de toda reversión**
   El código vuelve atrás en minutos y la migración aplicada no vuelve.

4. **La aprobación aporta trazabilidad, no detección**
   Y cuando es externa al equipo, la evidencia del sector muestra que alarga la entrega sin mejorar la estabilidad.

5. **Sin señal/alerta declarada el despliegue automático entrega el error más rápido**
   La ventana de observación y su umbral se escriben antes, igual que el quality gate.